Relatório Técnico de Entrega 9 — Integração Final, Segurança e Testes

Curso: Internet das Coisas III — IFMG Campus Ouro Branco

Professor: Charles Tim Batista Garrocho

Aluno: Daniel

Dispositivo: `esp32_temp` (ESP32 DevKit V1)

Projeto: Sistema de Alerta de Temperatura e Saúde Ambiental IoT

Data: Novembro / 2026

---

1. Resumo Executivo e Evolução Incremental (Entregas 1 a 9)

A **Entrega 9** conclui de forma incremental a integração de todas as camadas desenvolvidas ao longo da disciplina de **Internet das Coisas III**, validando a operação ponta a ponta entre o **ESP32 DevKit V1 (`esp32_temp`)**, a rede **Wi-Fi**, o **Broker MQTT (Eclipse Mosquitto 2.0)**, o **Gateway Node.js/TypeScript**, o banco em memória **Redis 7**, a **API RESTful** e o **Sistema Web em React 18**.

Além da operação em tempo real com telemetria multi-sensorial e controle remoto bidirecional confirmado via `ACK/state`, esta entrega consolida as **práticas de segurança** (isolamento de credenciais em `secrets.h` e `.env`, validação estrita de payloads e comandos com **Zod**, e restrição de acesso via **HTTP Basic Auth**), o **tratamento de falhas e recuperação automática de comunicação** (reconexão não-bloqueante no ESP32, *Last Will and Testament — LWT*, fallback em memória no backend e timeouts HTTP `503`/`504`) e a execução completa do **plano de testes funcionais e de integração (Vitest 11/11)** acompanhado de **15 evidências visuais reais**.

Entrega	Camada / Marco Incremental Consolidado	Artefatos Principais
Entrega 1	Prototipação Física no ESP32, Leitura de Sensores e Atuação Local (`DHT22`, `Som`, `BH1750`, `ST7789`)	`docs/entrega-1/`, `firmware/src/main.cpp`
Entrega 2	Conectividade e Resiliência Wi-Fi (`millis()`) e Isolamento de Credenciais (`secrets.h`)	`docs/entrega-2/`, `firmware/include/secrets.h`
Entrega 3	Comunicação MQTT (Mosquitto), Namespace Oficial IFMG, QoS, Retain, LWT e Controle Remoto	`docs/entrega-3/`, `mosquitto/config/mosquitto.conf`
Entrega 4	Gateway MQTT (Node.js/TypeScript), Subscrição Wildcard e Validação Estrita com Zod	`docs/entrega-4/`, `backend/src/mqtt/client.ts`
Entrega 5	Persistência no Redis, Histórico de Telemetria por Dia e Controle de Presença (`lastSeen`)	`docs/entrega-5/`, `backend/src/storage/kv_store.ts`
Entrega 6	Comunicação Bidirecional e Confirmação de Comandos pelo ESP32 (`ACK/state`)	`docs/entrega-6/`, `backend/src/mqtt/client.ts`
Entrega 7	API RESTful, Consultas ao Redis (`/state`, `/presence`, `/history`) e Erros HTTP (`400..504`)	`docs/entrega-7/`, `backend/src/api/routes.ts`

Entrega	Camada / Marco Incremental Consolidado	Artefatos Principais
Entrega 8	Sistema Web React 18 de Monitoramento em Tempo Real, Gráficos e Controle Remoto com ACK	`docs/entrega-8/`, `frontend/src/App.tsx`
Entrega 9	**Integração Final Ponta a Ponta, Segurança, Resiliência e Testes E2E**	`docs/entrega-9/`, `backend/tests/e2e.test.ts`

2. Tabela de Conformidade dos 13 Produtos Esperados (Entrega 9)

#	Produto Esperado	Status	Implementação Técnica e Evidência Visual no Relatório
1	**Integração completa (ESP32, MQTT, Gateway, Redis, API e Web)**	**100% Atendido**	Stack Docker (`4 containers Up/Healthy`) + ESP32 real na rede (Seção 3.2)
2	**Telemetria ponta a ponta funcionando**	**100% Atendido**	Publicação contínua em `.../telemetry` → Redis → API → UI Web (Seções 3, 4 e 8.1)
3	**Consulta de estado atual e histórico**	**100% Atendido**	Endpoints `GET /state`, `GET /presence` e `GET /history` com prints JSON (Seção 4.1)
4	**Controle remoto do atuador**	**100% Atendido**	Endpoint `POST /api/devices/:id/command` acionando LED (GPIO 12) e Relé (GPIO 14)
5	**Confirmação do estado do atuador pelo ESP32**	**100% Atendido**	Publicação retida em `.../state` (`ACK`) confirmada antes do `HTTP 200 OK` (Seções 4.2 e 8.2)
6	**Tratamento de falhas básicas e recuperação**	**100% Atendido**	Reconexão Wi-Fi/MQTT via `millis()`, LWT, Fallback Redis e `HTTP 503/504` (Seção 6)
7	**Proteção das credenciais do projeto**	**100% Atendido**	`firmware/include/secrets.h` e `.env` no `.gitignore` + `HTTP Basic Auth` (Seção 5)
8	**Validação dos comandos**	**100% Atendido**	`CommandPayloadSchema` (Zod) rejeitando payloads inválidos com `HTTP 400` (Seções 4.2 e 5.3)
9	**Plano de testes executado com evidências**	**100% Atendido**	Suíte automatizada Vitest (`11/11 PASS`) + Ensaios de falha F1–F3 (Seção 7)
10	**Código-fonte completo e organizado**	**100% Atendido**	Estrutura modular em `firmware/`, `mosquitto/`, `backend/`, `frontend/` e `docs/` (Seção 9)
11	**Diagrama final da arquitetura**	**100% Atendido**	Diagrama visual completo das 4 camadas e fluxos de dados (Seção 3.1)
12	**Documentação técnica final**	**100% Atendido**	Este documento técnico consolidado + `ARCHITECTURE.md` + `README.md`
13	**Demonstração prática da solução IoT completa**	**100% Atendido**	15 capturas reais dos endpoints JSON, terminais, testes e todas as visões Web (Seções 3 a 8)

3. Diagrama Final da Arquitetura e Evidência da Stack Integrada

3.1 Diagrama Visual da Arquitetura IoT em 4 Camadas

A arquitetura final conecta a **Camada de Borda (ESP32 DevKit V1)**, a **Camada de Mensageria MQTT (Eclipse Mosquitto 2.0)**, a **Camada de Gateway e Persistência (Node.js/TypeScript + Redis 7)** e a **Camada de API REST e Sistema Web (Express + React 18)**:



Figura: Diagrama Final da Arquitetura IoT de 4 Camadas

3.2 Evidência de Integração: Docker Compose, Monitor Serial ESP32, Broker MQTT e Auditoria de Credenciais

A captura abaixo comprova simultaneamente:

1. Os **4 containers Docker** (`iot-mosquitto`, `iot-redis`, `iot-backend`, `iot-frontend`) em estado `Up (healthy)`.

2. O **Monitor Serial do ESP32** (`/dev/cu.usbserial-5B520923941 @ 115200 bps`) conectado ao Wi-Fi (`SSID: Luca`, `IP: 192.168.1.175`) e ao broker MQTT (`192.168.1.170:1883`), publicando disponibilidade (`online`, `Retain=true`), telemetria periódica e confirmando comandos remotos recebidos (`action=set target=led value=true`).

3. Os **Logs sincronizados do Mosquitto e do Gateway Node.js** persistindo telemetria e confirmação no Redis.

4. A **Auditoria do** `.gitignore` comprovando o isolamento de `firmware/include/secrets.h` e `.env`.

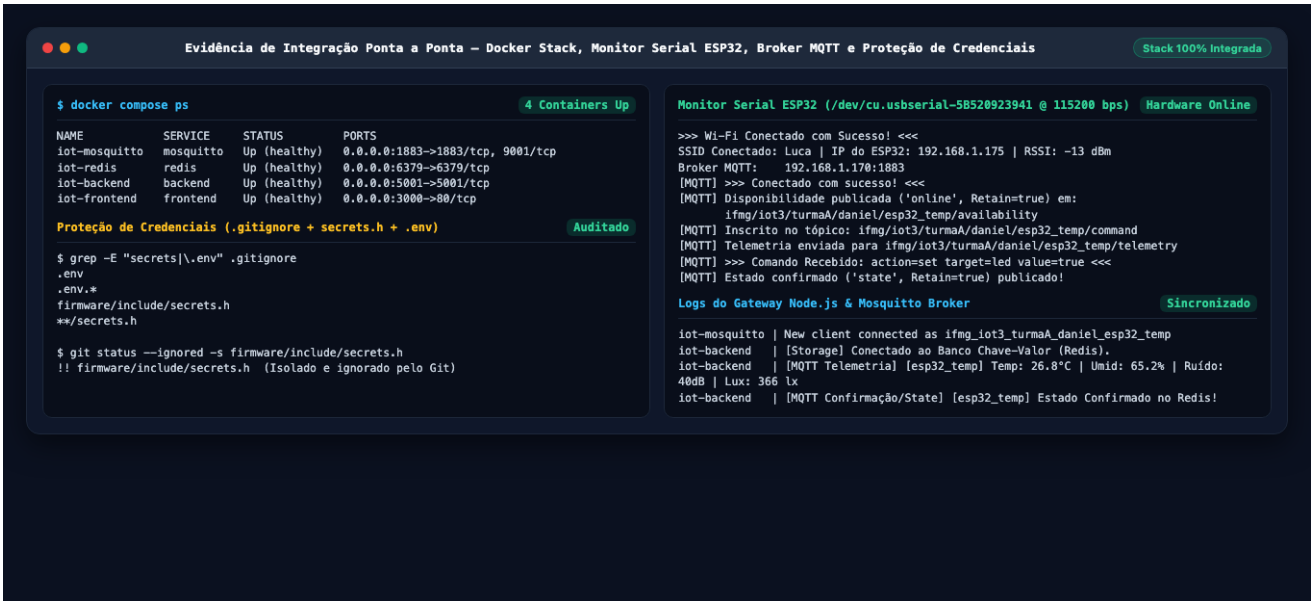


Figura: Evidência de Integração Ponta a Ponta: Docker Stack, Monitor Serial ESP32, Broker MQTT e Proteção de Credenciais

4. Evidências Reais dos Endpoints da API REST (Consultas, Comando com ACK e Erros HTTP)

4.1 Respostas Reais dos Endpoints de Consulta (`GET /health`, `/state`, `/presence` e `/history`)

O print a seguir apresenta as respostas JSON reais retornadas pelo container `iot-backend` (`http://localhost:5001/api`) alimentado pelo ESP32:

- `GET /api/health` **(**`HTTP 200 OK`**)**: confirma `mqttConnected: true` e `storageType: "redis"`.
- `GET /api/devices/esp32_temp/state` **(**`HTTP 200 OK`**)**: retorna o snapshot completo do dispositivo (`status: "online"`, `lastSeen`, `temp: 26.8`, `humidity: 65.1`, `noiseLevel: 40`, `lux: 400`, `lcdText: "AMBIENTE EM CONFORMIDADE"`, `displayType: "ST7789_SPI"`, `actuatorState: false`).
- `GET /api/devices/esp32_temp/presence` **(**`HTTP 200 OK`**)**: retorna especificamente a disponibilidade (`status: "online"`) e o carimbo `lastSeen`.
- `GET /api/devices/esp32_temp/history?limit=2` **(**`HTTP 200 OK`**)**: retorna a série temporal armazenada na lista `device:esp32_temp:history` no Redis.

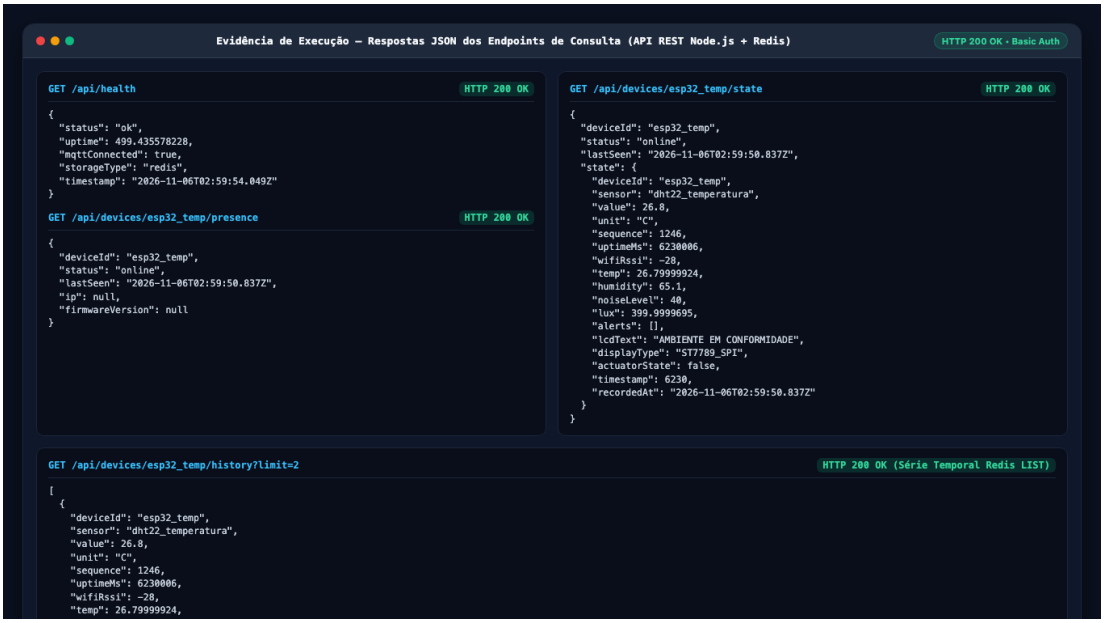


Figura: Evidência de Execução — Respostas JSON dos Endpoints de Consulta da API REST

4.2 Respostas Reais de Controle Remoto com Confirmação (`ACK`) e Erros HTTP (`401`, `404`, `400`)

O endpoint `POST /api/devices/:id/command` envia o comando ao tópico `ifmg/iot3/turmaA/daniel/esp32_temp/command` (QoS 1) e **aguarda a confirmação real do ESP32** no tópico `.../state` antes de atualizar o Redis e responder ao cliente:

- **Ligar Atuador (`{"action":"set","target":"led","state":true}` → `HTTP 200 OK`)**: retorna `success: true`, `commandId`, `actuatorState: true` e o timestamp exato `acknowledgedAt` confirmado pelo ESP32.
- **Desligar Atuador (`{"action":"set","target":"led","state":false}` → `HTTP 200 OK`)**: retorna `success: true`, `actuatorState: false` e `acknowledgedAt`.
- **Requisição sem Basic Auth (`GET /api/devices` sem cabeçalho `Authorization` → `HTTP 401 Unauthorized`)**: bloqueada pelo middleware `basicAuth` com `error: "UNAUTHORIZED"`.
- **Dispositivo Inexistente (`GET /api/devices/esp32_nao_cadastrado/state` → `HTTP 404 Not Found`)**: retorna `error: "NOT_FOUND"`.
- **Comando Malformado (`POST /api/devices/esp32_temp/command` com `action: "COMANDO_INVALIDO"` → `HTTP 400 Bad Request`)**: bloqueado pelo schema **Zod** (`CommandPayloadSchema`) com `error: "VALIDATION_ERROR"` e detalhamento do campo inválido.

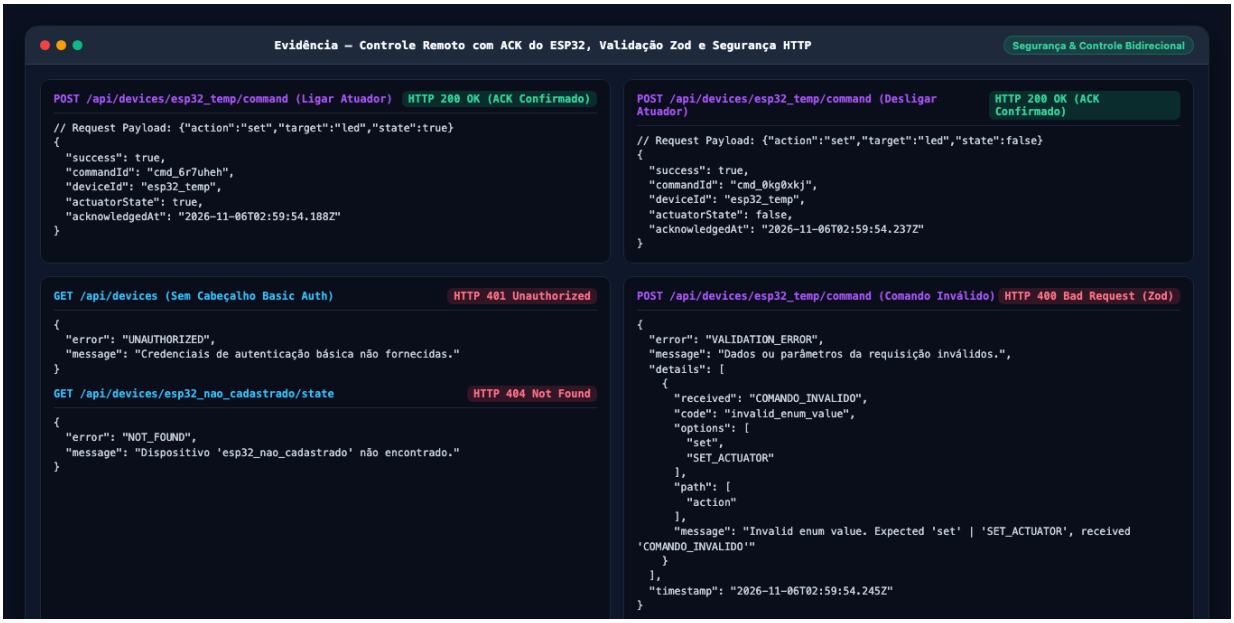


Figura: Evidência — Controle Remoto com ACK do ESP32, Validação Zod e Segurança HTTP

---

5. Práticas de Segurança Implementadas

5.1 Proteção de Credenciais no Firmware (`secrets.h` + `.gitignore`)

As credenciais de Wi-Fi (`WIFI_SSID`, `WIFI_PASSWORD`) e o endereço do broker (`MQTT_HOST`) nunca são gravados diretamente em `firmware/src/main.cpp`. Eles residem exclusivamente em `firmware/include/secrets.h` (ignorado pelo Git via `.gitignore`), enquanto o repositório disponibiliza apenas o modelo público `firmware/include/secrets.example.h`:

■ C++ / Firmware ESP32

```
// firmware/include/secrets.example.h (Versionado com segurança)
#pragma once
#define WIFI_SSID "NOME_DA_SUA_REDE_WIFI"
#define WIFI_PASSWORD "SENHA_DA_SUA_REDE_WIFI"
#define MQTT_HOST "192.168.1.100"
#define MQTT_PORT 1883
```

5.2 Variáveis de Ambiente e Isolamento de Configuração (`.env` + `config/env.ts`)

No backend e no Docker Compose, todas as credenciais e endereços de serviço (`API_USER`, `API_PASS`, `MQTT_BROKER_URL`, `REDIS_URL`, `PORT`) são carregados por variáveis de ambiente em `backend/src/config/env.ts`, mantendo arquivos `.env` fora do controle de versão.

5.3 Restrição de Acesso via HTTP Basic Auth (`backend/src/middleware/auth.ts`)

Todas as rotas de API sob `/api/devices` exigem autenticação `HTTP Basic Auth`. Requisições não autenticadas ou com credenciais incorretas recebem `HTTP 401 Unauthorized` imediatamente, impedindo que usuários não autorizados consultem dispositivos ou acionem o atuador remotamente.

5.4 Validação Estrita de Comandos e Payloads com Zod (`backend/src/schemas/payloads.ts`)

Todos os comandos recebidos pela API REST e todos os pacotes MQTT que chegam do broker são validados com **Zod** antes de qualquer processamento ou persistência:

■ TypeScript / Gateway Node.js

```
export const CommandPayloadSchema = z.object({
  action: z.enum(['set', 'SET_ACTUATOR', 'TOGGLE']).default('set'),
  target: z.enum(['led', 'relay', 'buzzer', 'all']).default('led'),
  state: z.boolean().optional(),
  value: z.boolean().optional(),
}).refine((data) => data.state !== undefined || data.value !==
    undefined, {
  message: "O comando deve informar 'state' (boolean) ou 'value'
      (boolean).",
});
```

---

6. Tratamento de Falhas Básicas e Recuperação da Comunicação

1. **Queda de Wi-Fi ou Broker MQTT no ESP32 (`firmware/src/main.cpp`)**:

- A função `checkConnectivity()` utiliza temporizadores não-bloqueantes baseados em `millis()` (`RECONNECT_INTERVAL_MS = 10000`). Enquanto o Wi-Fi ou o broker estiver indisponível, o ESP32 **não trava**: continua lendo sensores e atualizando os alertas locais no display ST7789. Assim que a rede retorna, reconecta automaticamente, republica `online` (Retain) em `.../availability` e reassina `.../command`.

2. **Desconexão Abrupta / Falha de Energia do ESP32 (`Last Will and Testament — LWT`)**:

- No handshake MQTT, o ESP32 registra no Mosquitto um **LWT** no tópico `ifmg/iot3/turmaA/daniel/esp32_temp/availability` com payload `"offline"` (`QoS 1`, `Retain = true`). Se o ESP32 perder alimentação elétrica, o broker publica `"offline"` automaticamente após o timeout do keepalive (`15s`), atualizando o Redis e exibindo o badge `Offline` na interface web.

3. **Indisponibilidade Transitória do Redis ou Broker no Backend (`kv_store.ts` e `error_handler.ts`)**:

- Caso o container Redis fique indisponível, `kvStore` ativa transparentemente um **fallback em memória (`Map`)**, mantendo a API funcional.

- Caso o broker MQTT esteja desconectado durante um comando, a API retorna `HTTP 503 Service Unavailable` (`MQTT_DISCONNECTED`).

- Caso o ESP32 esteja desligado e não confirme o comando (`ACK/state`) dentro de `5000 ms`, a Promise de confirmação expira com `HTTP 504 Gateway Timeout` (`ACTUATOR_ACK_TIMEOUT`), preservando o estado real no Redis sem gravação incorreta.

---

7. Plano de Testes Executado e Evidências

A suíte automatizada (`backend/tests/e2e.test.ts`, executada via **Vitest**) valida todas as regras de negócio, persistência Redis, confirmação `ACK` e códigos de erro HTTP:

ID	Caso de Teste Executado	Camadas Envolvidas	Resultado Esperado	Status
CT-01	Validação de payload de telemetria válido (`TelemetryPayloadSchema`)	Gateway + Zod	Payload aceito e normalizado	**PASS**
CT-02	Rejeição de telemetria inválida / corrompida	Gateway + Zod	Bloqueio seguro sem corromper banco	**PASS**
CT-03	Validação de presença (`online`/`offline` LWT)	MQTT + Gateway + Zod	Status interpretado corretamente	**PASS**

ID	Caso de Teste Executado	Camadas Envolvidas	Resultado Esperado	Status
CT-04	Validação de comandos REST HTTP (`CommandPayloadSchema`)	API REST + Zod	Aceita comandos válidos e rejeita inválidos	**PASS**
CT-05	Persistência e consulta de estado atual (`device:{id}:state`)	Gateway + Redis	Estado recuperado com precisão	**PASS**
CT-06	Persistência e consulta de histórico com filtro por data (`YYYY-MM-DD`)	Gateway + Redis (`LIST`)	Série temporal filtrada e ordenada	**PASS**
CT-07	Atualização de presença e carimbo `lastSeen` (`device:{id}:presence`)	Gateway + Redis	Transição `online`/`offline` registrada	**PASS**
CT-08	Validação de confirmação (`ACK/State`) publicada pelo ESP32	ESP32 + MQTT + Zod	Confirmação validada pelo schema	**PASS**
CT-09	Atualização do atuador no Redis **apenas após confirmação ACK**	Gateway + MQTT + Redis	Estado persiste somente após retorno do nó	**PASS**
CT-10	Consultas integradas via endpoints REST (`/state`, `/presence`, `/history`)	API REST + Basic Auth + Redis	`HTTP 200 OK` com JSON estruturado	**PASS**
CT-11	Códigos HTTP adequados para falhas (`401`, `404`, `400`, `503` e `504`)	API REST + Error Handler	Respostas padronizadas de erro HTTP	**PASS**

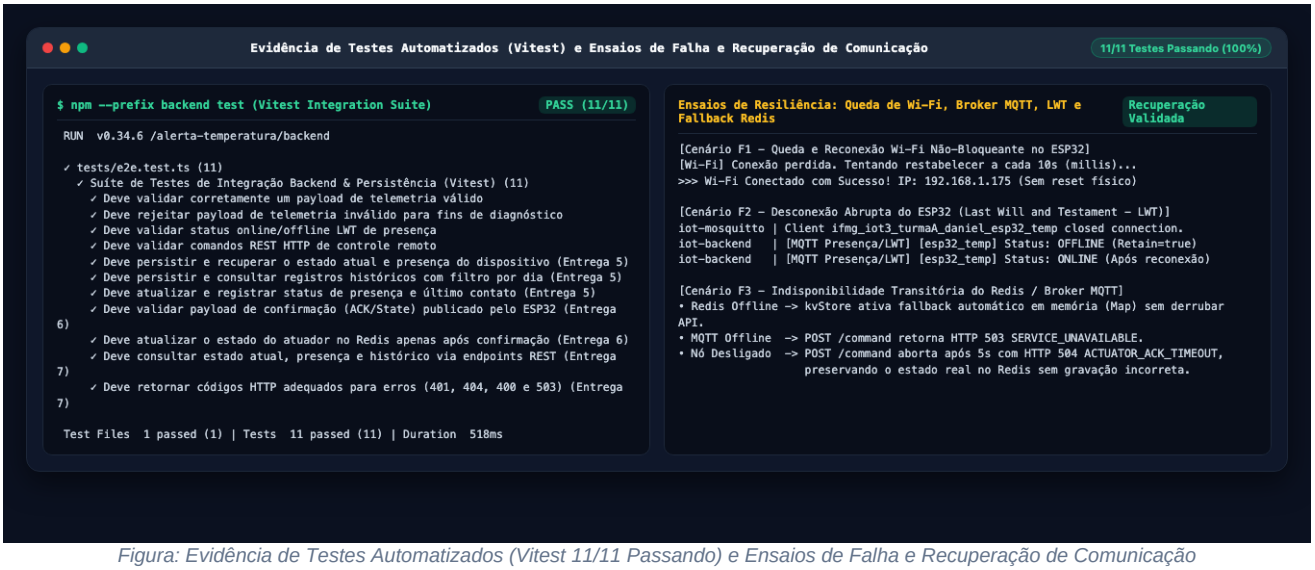


Figura: Evidência de Testes Automatizados (Vitest 11/11 Passando) e Ensaios de Falha e Recuperação de Comunicação

8. Demonstração Prática Completa no Sistema Web (Monitoramento, Gráficos, Histórico e Controle Remoto)

8.1 Presença Online, Último Contato e Estado Atual dos Sensores

O cabeçalho apresenta o seletor do dispositivo `esp32_temp (online)`, o badge verde `Online` com o horário exato do último contato (`lastSeen`) e os 4 cartões de telemetria em tempo real (**Temperatura** `26.8 °C`, **Umidade Relativa** `65.2 %`, **Nível de Ruído** `40 dB` e **Luminosidade** `383.3 lx`):



Figura: Monitoramento em Tempo Real: Presença Online, Último Contato e Cartões de Sensores

8.2 Controle Remoto do Atuador Antes e Depois da Confirmação (ACK) pelo ESP32

1. **Estado Inicial (`DESLIGADO`)**: O simulador da tela TFT IPS ST7789 exibe `ATUADOR: DESLIGADO` e o painel de controle remoto apresenta o botão `■ Ligar Atuador`:

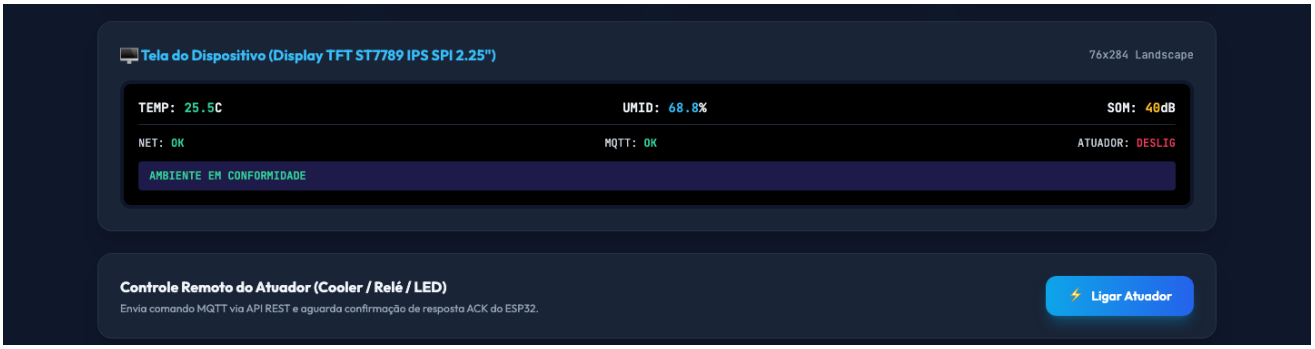


Figura: Display ST7789 e Controle Remoto com Atuador Desligado

2. **Estado Confirmado pelo ESP32 (`LIGADO — Confirmado via ACK`)**: Após clicar no botão de acionamento, o comando é publicado via MQTT, executado fisicamente pelo ESP32 (GPIO 12/14) e confirmado em `.../state`, atualizando o display para `ATUADOR: LIGADO` e o indicador verde `LIGADO (Confirmado via ACK)`:

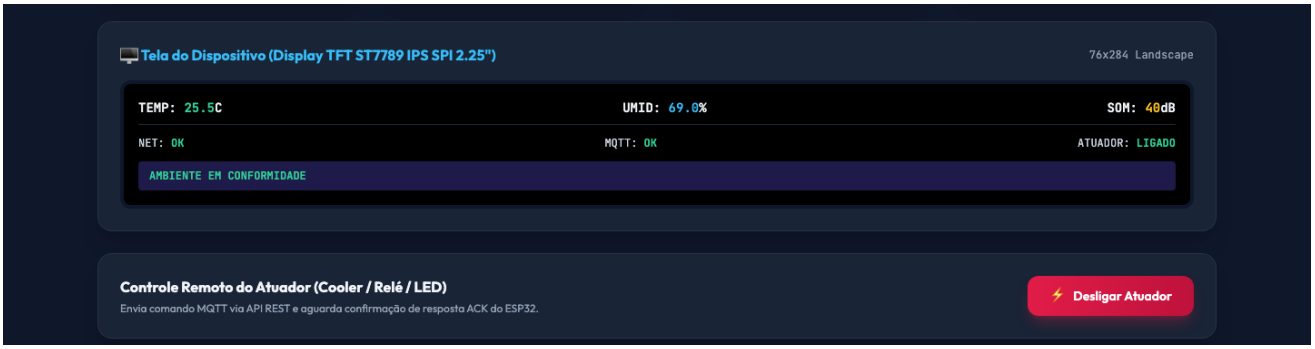


Figura: Atuador Acionado Remotamente e Confirmado via ACK pelo ESP32

8.3 Gráficos Históricos das 4 Variáveis Sensoriais e Escala Temporal (`Dia`, `Semana`, `Mês`)

As capturas a seguir demonstram o gráfico temporal (`HistoryChart.tsx`) em cada uma das **4 abas sensoriais** (`Temperatura`, `Umidade`, `Ruído` e `Luminosidade`) com escala temporal proporcional às leituras no filtro **Dia**, além da visualização com o filtro **Mês**:



Figura: Gráfico Histórico — Aba Temperatura (°C) com Escala Temporal por Dia



Figura: Gráfico Histórico — Aba Umidade (%) com Escala Temporal por Dia



Figura: Gráfico Histórico — Aba Ruído (dB) com Escala Temporal por Dia

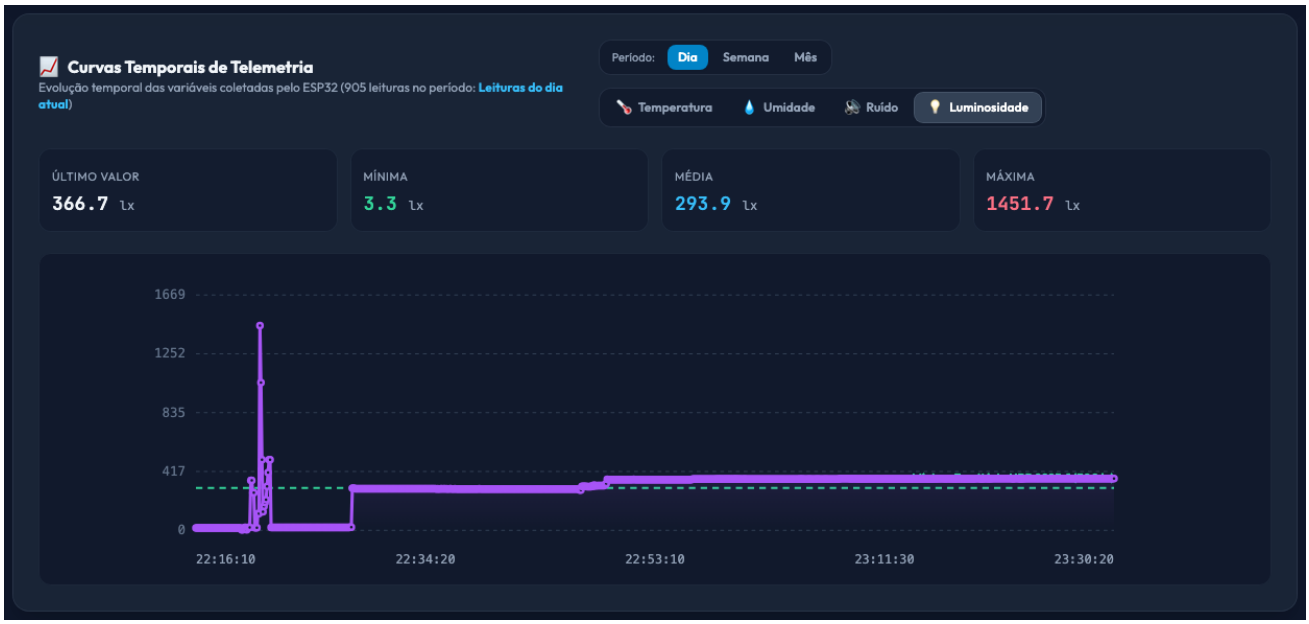


Figura: Gráfico Histórico — Aba Luminosidade (lx) com Escala Temporal por Dia



Figura: Gráfico Histórico — Filtro de Escala Temporal Mês (DD/MM HH:MM)

8.4 Tabela de Histórico de Telemetria, Filtro por Data e Paginação

A tabela de registros históricos (`HistoryTable.tsx`) exibe o carimbo de data/hora, sequência (`#seq`), valores dos 4 sensores, RSSI Wi-Fi e estado do atuador, contando com filtro por dia (`YYYY-MM-DD`), paginação e exportação para CSV:

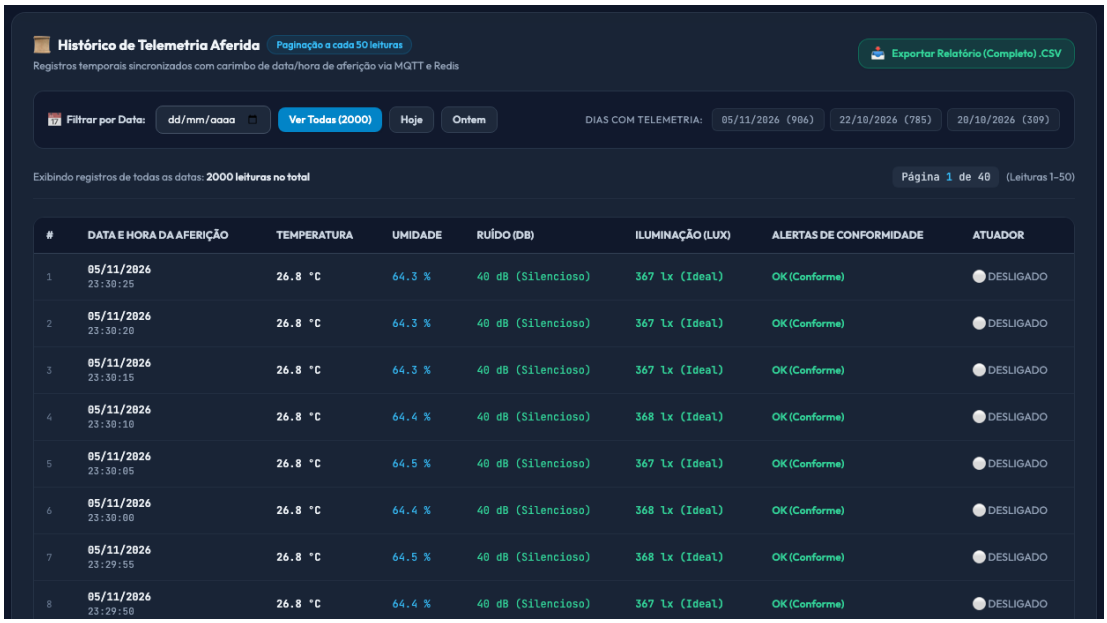


Figura: Tabela de Histórico de Telemetria Completa

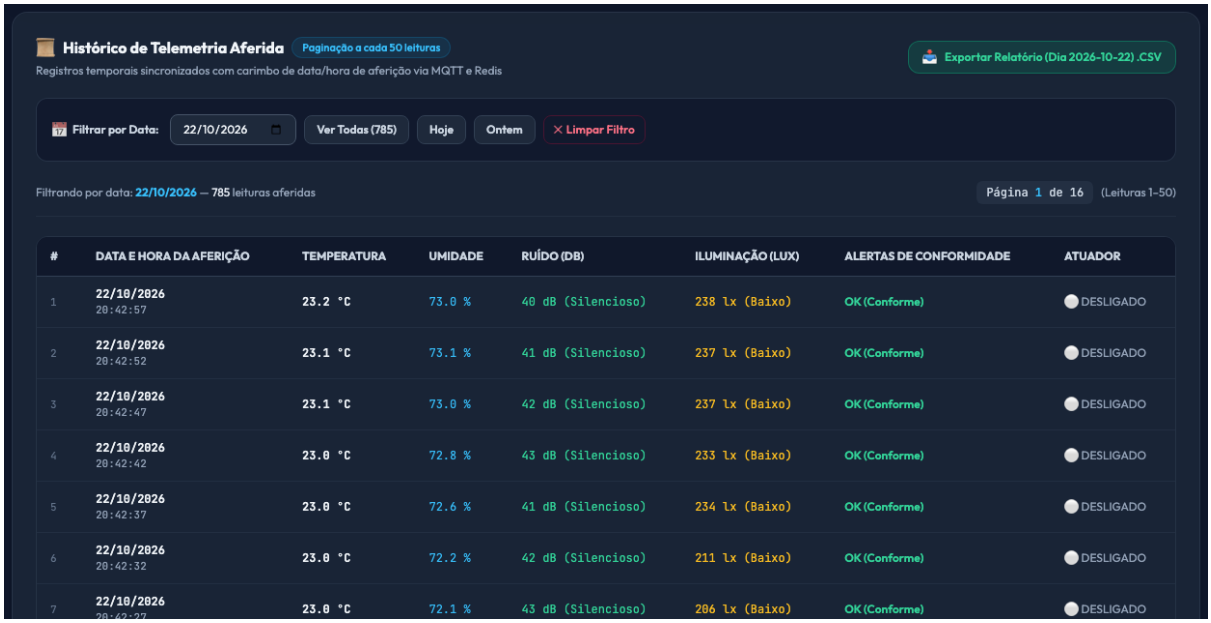


Figura: Tabela de Histórico Filtrada por Data



Figura: Tabela de Histórico Navegando na Página 2 da Paginação

9. Organização Final do Código-Fonte e Guia de Execução

■ Log de Execução / Diagnóstico

```
alerta-temperatura/
■■■ ARCHITECTURE.md                  # Especificação arquitetural e
    contratos MQTT/REST/Redis
■■■ DOCKER.md                        # Guia de containers, portas e
    volumes Docker Compose
■■■ README.md                        # Documentação geral e guia passo a
    passo de execução
■■■ docker-compose.yml               # Orquestração dos 4 serviços
    (mosquitto, redis, backend, frontend)
■■■ firmware/                        # Firmware Embarcado C++
    (PlatformIO / ESP32 DevKit V1)
■   ■■■ platformio.ini               # Configuração da placa esp32dev e
    dependências
■   ■■■ include/secrets.example.h    # Template seguro de credenciais
    Wi-Fi e Broker MQTT
■   ■■■ src/main.cpp                 # Leitura DHT22/Som/BH1750, Display
    ST7789, Wi-Fi/MQTT LWT e ACK
■■■ mosquitto/
■   ■■■ config/mosquitto.conf        # Configuração do broker Eclipse
    Mosquitto 2.0 (1883 / 9001)
■■■ backend/                         # Gateway MQTT + Persistência Redis
    + API RESTful (Node.js/TypeScript)
■   ■■■ src/config/env.ts            # Carregamento seguro de variáveis
    de ambiente (.env)
■   ■■■ src/schemas/payloads.ts      # Schemas Zod (Telemetry,
    Availability, StateAck, Command)
■   ■■■ src/mqtt/client.ts           # Consumidor MQTT, publicação de
    comandos e espera de confirmação ACK
■   ■■■ src/storage/kv_store.ts      # Persistência Redis 7 com fallback
    automático em memória
■   ■■■ src/middleware/              # basicAuth (HTTP 401) e
    errorHandler global (HTTP 400..504)
■   ■■■ src/api/routes.ts            # Endpoints REST (/health,
    /devices, /state, /presence, /history, /command)
■   ■■■ tests/e2e.test.ts            # Suíte automatizada Vitest (11
    testes de integração ponta a ponta)
■■■ frontend/                        # Sistema Web SPA (React 18 +
    TypeScript + TailwindCSS + Vite)
■   ■■■ src/
■       ■■■ App.tsx                  # Dashboard principal com polling
    autenticado e controle remoto
■       ■■■ components/              # MetricCard, DeviceBadge,
    ST7789Display, ActuatorControl, HistoryChart, HistoryTable
■■■ docs/                            # Relatórios técnicos das Entregas
    1 a 9 (MD + PDF) + Evidências PNG
```

Comandos Rápidos para Reprodução e Demonstração

■ Terminal / Comandos de Execução

```
# 1. Inicializar toda a stack (Mosquitto, Redis, Backend e Frontend)
docker compose up -d --build

# 2. Compilar e gravar o firmware no ESP32 conectado via USB
/Users/daniel/.platformio_env/bin/pio run -d firmware -t upload

# 3. Executar a suíte de testes automatizados de integração (Vitest)
npm --prefix backend test -- --run

# 4. Acessos locais:
# Sistema Web: http://localhost:3000
# API Health:  http://localhost:5001/api/health
```